JIOHOTSTAR

Senior DevOps Engineer

Interview Preparation Guide

3 Rounds | 15 Questions | Complete Answer Frameworks

Streaming at Scale · K8s · Cloud · RCA · Fire Drills · Leadership

Round 1 Streaming & K8s 45 mins	Round 2 RCA & Fire Drills 75 mins	Round 3 Leadership & Culture 30 mins

■ **Read This First:** This is a Senior DevOps role requiring 5–7 years of production-scale experience. Your credibility is built on **depth of thinking and frameworks**, not fabricated war stories. Use ApnaKart and Hisan Labs wherever possible. Own the gap in Round 3 — interviewers respect self-awareness.

ROUND 1 — Streaming at Scale, K8s, Cloud & Linux (45 mins) 1

Q1 **How would you design auto-scaling for 50M+ concurrent viewers across multi-cluster K8s without over-provisioning?**

The trap: Saying 'just set HPA on CPU/memory.' CPU lags 30–60s behind stream spikes — by the time HPA reacts, users are buffering.

Scale on leading indicators, not lagging ones:

- **KEDA** (Kubernetes Event-Driven Autoscaler) on custom metrics: concurrent stream count from CDN telemetry, Kafka consumer lag, viewer connection rate — all available 10–30s before CPU spikes.
- **Predictive pre-scaling:** Use Prometheus historical data + CronHPA to pre-scale before known events (IPL 7:30 PM → start scaling at 6:45 PM).
- **Multi-cluster strategy:** Separate clusters per function — ingest, transcoding, session/auth, CDN-origin. Scale each independently.
- **Node pool segmentation:** Spot-heavy pools for stateless transcoding; on-demand reserved for stateful session stores.
- **Cluster Autoscaler tuning:** Set `--scale-down-utilization-threshold=0.8` during event windows. Pre-warming cost is lower than a 30-second cold start during a live final.
- **VPA in recommendation mode:** Ensure `requests` are accurately set — HPA accuracy depends entirely on this.

■ Anti-over-provisioning: Set `--scale-down-delay-after-add=10m` to prevent flapping. Spot instance interruption handlers drain gracefully before node removal.

Q2 **New region needs to spin up instantly during IPL final. How do you pre-warm nodes with zero cold-start impact?**

Cold-start has **three layers** — address each separately:

Node Level (AWS)

- **Warm Pools** in Auto Scaling Groups: instances pre-launched and stopped, available in <30s vs 3–4 minutes cold.
- **Karpenter** with pre-configured NodeTemplates for rapid provisioning.
- Pre-pull container images via DaemonSet 30 mins before event (`ImagePullPolicy: IfNotPresent` only helps if image is already cached).

Pod Level

- **Placeholder pods:** Deploy lightweight pods with the same resource requests as production pods. They hold node capacity warm. Replace on demand — scheduling is instant.
- `startupProbe` with generous `initialDelaySeconds` to prevent premature liveness killing during JVM warmup.
- Pre-warm application caches via init containers hitting internal health endpoints.

Traffic Level

- Istio `VirtualService` weight-based routing: shift 0% traffic to new region initially, ramp only after readiness probes pass across all pods.
- Use **Global Accelerator** or Anycast — not DNS — for near-instant regional traffic switching without TTL delays.

Q3 **Use Envoy + Istio to route live streams differently from VOD without service restarts.**

Key point: VirtualService and DestinationRule changes are applied **without pod restarts** — Istio pushes config to Envoy sidecars via xDS protocol dynamically.

VirtualService — split by header:

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
spec:
  http:
  - match:
    - headers:
        x-content-type:
          exact: live-stream
    route:
    - destination:
        host: stream-service
        subset: live     # LEAST_CONN, fail-fast, no retries
  - route:
    - destination:
        host: stream-service
        subset: vod      # ROUND_ROBIN, retries acceptable
```

	Live Streams	VOD
Load Balancing	LEAST_CONN	<font name="Courier" color="#C7254E" size="8.5">ROUND_ROBIN</fo
Connect Timeout	100ms (fail fast)	2s
Retries	None (retry = lag spike)	3 attempts
Circuit Breaker	2 consecutive errors	5 consecutive errors

Q4 **Multi-zone pod affinity/anti-affinity to ensure node failure doesn't impact regional streaming SLAs.**

Don't use podAntiAffinity alone — respected at scheduling time but doesn't rebalance on node failures. Use **topologySpreadConstraints**:

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule   # hard guarantee, not soft
  labelSelector:
    matchLabels:
      app: stream-ingest
```

- **maxSkew: 1** → pods distributed within 1 replica difference across zones.
- **whenUnsatisfiable: DoNotSchedule** → hard guarantee. Scheduler refuses to place pods if it would violate spread.

- **podAntiAffinity** at host level → no two critical pods on same node (`requiredDuringSchedulingIgnoredDuringExecution`).
- **PodDisruptionBudget:** `minAvailable: 67%` — zone failure leaves you with 2/3 capacity still serving.

Q5 **Monitor HPA scaling decisions in real-time and detect if the metrics server is lagging.**

Real-time commands:

```
kubectl get hpa -w -n streaming
kubectl describe hpa stream-ingest-hpa   # check Conditions block
kubectl get apiservice v1beta1.metrics.k8s.io -o yaml   # check Available: False
```

Prometheus alerts to configure:

- `kube_horizontalpodautoscaler_status_condition{condition='ScalingActive',status='false'}` → alert immediately
- `kube_horizontalpodautoscaler_status_desired_replicas != kube_horizontalpodautoscaler_status_current_replicas` → scaling lag
- metrics-server P99 response > 10s → HPA decisions become stale

■ Mitigation: Deploy KEDA alongside HPA for redundancy. KEDA pulls from Prometheus directly, bypassing metrics-server entirely.

Q6 **Readiness/liveness probe configs to catch buffering/lag in stream-processing microservices before users notice.**

Common mistake: Using same endpoint for both probes. Readiness fail = pulled from LB. Liveness fail = container killed. These are different actions.

```
startupProbe:           # takes over during init — critical for JVM apps
  httpGet:
    path: /health/startup
  failureThreshold: 30
  periodSeconds: 5      # 150s total startup window

livenessProbe:
  httpGet:
    path: /health/live  # thread alive, JVM not deadlocked
  periodSeconds: 10
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /health/ready # checks: Kafka assigned, buffer lag < N, deps responding
  periodSeconds: 5
  failureThreshold: 2   # pull from LB faster than liveness kills
```

/health/ready must check:

- Kafka consumer group assignment confirmed
- Buffer depth below configured maximum
- Last message processed within N seconds
- Connection pool to Redis/DB healthy

Q7 **A kube-proxy update rolls out mid-match. What's your network rollback plan to avoid packet drops?**

First question to raise: Why is a DaemonSet rolling out during a live match? Your change freeze process failed — mention this in the answer.

Immediate rollback:

```
kubectl rollout undo daemonset/kube-proxy -n kube-system
kubectl rollout status daemonset/kube-proxy -n kube-system

# Detect on affected nodes:
iptables -L | wc -l    # kube-proxy bug can flush/corrupt rules
```

- **DaemonSet strategy:** `maxUnavailable: 1` — never more than 1 node updating simultaneously.
- **Pre-update:** Cordon node (`kubectl cordon `), drain traffic, update, uncordon.
- **Alert:** Watch `node_network_receive_drop_total` and `kube_node_status_condition{condition=NetworkUnavailable}`.
- **Long-term:** Migrate to Cilium eBPF (kube-proxy replacement) — no iptables dependency, faster rule propagation.

ROUND 2 — RCA, Fire Drills & Streaming Chaos (75 mins) 2

Q1 **Playback failures spike for only 3% of APAC users. CPU, memory, pods look fine. Walk through your triage.**

'Pods look fine' is the misdirection. The problem is almost never where dashboards look clean. Start with what the error actually is.

Step 1 — Error codes from player telemetry

What error? 403 = DRM/auth, 404 = CDN miss, 5xx = origin, timeout, codec error. This narrows the problem space by 70% before touching a single kubectl command.

Step 2 — CDN layer

Check APAC edge PoP health (Singapore, Mumbai, Tokyo). Cache HIT/MISS ratio. A sudden MISS surge drives origin overload → latency spike.

Step 3 — DRM/Token auth

Is the auth service APAC endpoint responding? Token validation failures appear as playback failures, not pod crashes.

Step 4 — Network path

Run traceroute/mtr from APAC edge to origin. BGP route change? ISP congestion? TLS certificate expiry or SNI mismatch on APAC load balancer?

Step 5 — Device/OS segmentation

Is it 3% of users or 3% of sessions? Mobile/TV/web split? A codec update or manifest format change can break only specific clients.

Step 6 — DNS

TTL-cached stale resolution to a deprecated IP in APAC region.

■ Wrong first moves: Restarting pods, checking Kubernetes events, looking at node metrics. Infrastructure is already telling you it's fine.

Q2 **Kafka ingestion lags by 2 minutes during traffic surge. Producers fine, consumers idle. Debug path.**

Consumers idle + lag growing = consumers are not polling. Three most likely causes:

1. Consumer group rebalancing (most common)

```
kafka-consumer-groups.sh --bootstrap-server <broker> \
  --describe --group <group>
# Look for STATE: PreparingRebalance or CompletingRebalance
# During rebalance, ALL consumption stops
```

Caused by: `max.poll.interval.ms` exceeded (consumer takes too long between polls → broker kicks it out → rebalance loop).

Fix: Increase `max.poll.interval.ms`, decrease `max.poll.records` so each poll completes faster.

2. Processing deadlock

Consumer is pulling messages but the processing thread is stuck (DB timeout, lock contention). Consumer appears 'idle' because it's not committing offsets. Check downstream service metrics.

3. Consumer coordinator (broker) under load

One specific broker is the group coordinator. If that broker is under load, heartbeat timeouts trigger repeated rebalances. Check that broker's metrics specifically.

Q3 **Sudden tail latency on Redis-based stream session store during a Champions League match.**

Debug commands first:

```
redis-cli SLOWLOG GET 25          # commands exceeding threshold
redis-cli INFO stats              # keyspace_hits, evicted_keys
redis-cli INFO memory             # used_memory vs maxmemory
redis-cli INFO clients            # blocked_clients count
redis-cli --latency-history       # real-time latency sampling
redis-cli --hotkeys               # requires allkeys-lfu policy
```

Hot key (most likely during a match)

One session key getting millions of ops/sec. Redis is single-threaded — one hot key blocks everything. Fix: shard it across N Redis instances using `session: % N`, or use local in-process cache (Caffeine) fronting Redis with short TTL.

Memory pressure + eviction

If `used_memory` ≈ `maxmemory`, Redis evicts on every SET, blocking requests. Fix: increase memory or switch to `allkeys-lru`.

RDB snapshot triggered (BGSAVE)

Background save forks the process — CoW memory pressure causes latency spikes. Disable RDB during event windows; use AOF-only or disable persistence for ephemeral session data.

Large value keys

A session object grown to MB-range. Check with `DEBUG OBJECT` .

Q4 **HPA refuses to scale even though Prometheus shows CPU > 90%. Root cause and fix.**

Run `kubectl describe hpa` first. The `Conditions` block tells you exactly why scaling isn't happening.

No CPU `requests` defined on pods

HPA calculates utilization as `actual_cpu / requested_cpu`. Without requests, the denominator is undefined. HPA refuses to scale or behaves unpredictably. Fix: always set resource requests.

Already at `maxReplicas`

Check the HPA spec. If `maxReplicas: 5` and you're at 5, HPA won't scale regardless. You'll see `AbleToScale: False` with reason `TooManyReplicas`. Increase maxReplicas.

metrics-server stale

HPA polls every 15s. Restart metrics-server. Verify `APIService` status.

PodDisruptionBudget blocking

PDB `minAvailable` may prevent new pods if existing ones are in disrupted state.

Namespace ResourceQuota exhausted

Run `kubectl describe quota -n` . CPU quota maxed = new pods can't be scheduled.

Cluster Autoscaler hasn't run yet

New pods are Pending — scheduled but no node has capacity. HPA scaled; Kubernetes just can't place pods yet. Check `kubectl get pods | grep Pending`.

Q5 **NAT gateway costs double in 24 hours during a live series. No infra changes were made.**

'No infra changes' is the clue — this is almost always a software/config change or unexpected traffic pattern.

Debug path:

```
# VPC Flow Logs — identify source of outbound traffic
# Filter: srcaddr = private IP range, direction = egress
# Group by srcaddr to find culprit instance/pod

# CloudWatch: BytesOutToDestination metric on NAT gateway
# Cost Explorer: drill down to resource-level granularity
```

Missing VPC Endpoints (#1 silent cost killer)

New microservice deployed without VPC endpoints for S3, ECR, Secrets Manager, SQS — all traffic routes through NAT instead of private AWS backbone.

Container image pulls through NAT

ECR endpoint missing → every pod start pulls from public ECR through NAT. During a 50-node scale-out, this compounds fast.

Misconfigured logging/metrics

New Datadog agent config pushing full trace payloads instead of sampled, or a log shipper sending duplicates externally.

Compromised workload

A pod making sustained outbound calls to external IPs (cryptominer, data exfiltration). Flow Logs will show high-volume connections to unusual IPs.

■ Fix: VPC endpoints for all AWS services (always cheaper than NAT at scale). Set up daily cost anomaly alerts on NAT gateway metrics.

ROUND 3 — Leadership, Reliability Culture & Scaling Influence (30 mins) 3

Q1 **How do you build a culture where latency SLOs are enforced like uptime SLAs in a streaming org?**

Core problem: Teams treat latency as a 'performance concern,' not a 'reliability concern.' Fix the framing first.

SLOs with error budgets, not thresholds

'P99 < 200ms, 99.5% of the time' gives teams a budget to spend. Error budget burn rate alerts (5% budget burned in 1 hour) create urgency before breach — not after.

Latency gates in CI/CD

Run performance regression tests (k6, Gatling) on every PR. A 15% p99 regression fails the pipeline. Latency becomes a first-class engineering concern, not a post-deployment surprise.

Joint ownership with product

SLO breaches appear in sprint reviews. Engineering and PM own the error budget together. When budget is low, feature velocity slows — product feels the pressure.

Blameless postmortems with latency RCA

Every latency incident gets the same weight as an outage in the postmortem process.

Visibility

Latency dashboards visible to all engineers — not just 'is it up' but p50/p95/p99 per service, live.

Q2 **Ship multi-region failover for live events in 2 weeks with no DNS-based routing allowed.**

No DNS = no Route53 failover, no GSLB. You need **network-layer or application-layer** routing.

Option 1 — AWS Global Accelerator (recommended answer for AWS)

Routes at the AWS network layer using Anycast IPs. Failover in <30s without DNS TTL delays. Two static Anycast IPs → both regions registered as endpoints → health-check-based routing at the network layer. **This is the answer for AWS.**

Option 2 — CDN-based routing

CloudFront/Akamai/Fastly can route origin requests based on health checks at the CDN edge — no DNS change needed by clients.

Option 3 — Application-level region selection

Embed region logic in the client player SDK. Primary region URL hardcoded, fallback URL for secondary. On connection failure/timeout, player retries secondary. Crude but achievable in 2 weeks.

2-week execution plan:

- **Week 1:** Set up Global Accelerator, configure both regions as endpoints, configure health checks, test failover manually.
- **Week 2:** Load test failover behavior, update runbooks, verify session affinity behavior across regions (stateful sessions need replication or graceful drop strategy).

Q3 **How would you simulate chaos in a streaming pipeline without risking real user impact?**

Principle: Chaos on production is acceptable only when blast radius is controlled and observable. Never chaos during peak hours.

Shadow traffic + chaos

Mirror 5% of live traffic to a staging cluster via Istio `mirror:` in VirtualService. Inject chaos there — real traffic shape, zero user impact.

Synthetic user population

Maintain a pool of test accounts generating realistic streaming sessions 24/7. All chaos experiments target only synthetic users, excluded via feature flag or user-ID range.

Istio fault injection

No new tooling needed. Inject HTTP errors and delays via VirtualService `fault:` block scoped to % of traffic only.

LitmusChaos / Chaos Mesh

Node kill, pod delete, network partition — scoped to dedicated chaos namespaces or non-production node pools.

Canary + chaos

Deploy new version to 5% canary. Inject chaos on canary tier. Worst case: 5% of users on a canary already being validated.

Game Days

Scheduled (announced) experiments during low-traffic windows (3–6 AM). Full engineering team on standby. Never during IPL.

Q4 **How do you justify infra costs for pre-warmed scaling capacity to executives before a major sports event?**

Executives don't care about node pools. They care about revenue and risk. Frame it as insurance math.

■ "IPL final → ~20M concurrent viewers. Revenue per viewer (ad + subscription) during that window = ■X. 5-minute degradation affecting 10% of viewers = ■Y in direct revenue loss + subscriber churn. Pre-warming 200 nodes for 3 hours = ■Z. Z is less than Y by a factor of 10."

Supporting data points to bring:

- Last year's incident cost — tie a specific number to a specific event.
- Pre-warming as % of total event infra spend — typically under 15%.
- **Cost reduction lever:** 70% spot + 30% on-demand for pre-warmed pool drops cost significantly while maintaining availability.
- Time-bounded cost: pre-warmed capacity is scale-down protected for 4 hours, then normal autoscaling resumes. Not a permanent overhead.
- **Competitive framing:** "During the 2023 IPL, a competing platform experienced buffering for 8 minutes. Their app store rating dropped 0.4 points that week. We can avoid that for ■Z."

Interview Strategy — Owning the Experience Gap

1. Own the gap in Round 3 — upfront:

"My hands-on production experience is at internship scale, but I've deeply studied these failure modes through ApnaKart and architectural research." Interviewers respect self-awareness over fabricated experience.

2. Use ApnaKart as your production anchor in Rounds 1–2:

Map your KEDA/HPA configs, monitoring setup, Kafka integration, and Kubernetes manifests to each question. Even at smaller scale, the pattern of thinking is what's being evaluated.

3. Ask smart diagnostic questions:

In Round 2 RCAs, asking "what were the error codes in player telemetry?" before diving into infrastructure signals senior-level systems thinking. More impressive than a memorized answer.

4. Language that signals ownership:

"I would investigate..." / "The first thing I'd check is..." / "In my ApnaKart setup, I handled this by..." — not passive, not vague.